Design and Evaluation of a Low-Cost, Two-Node High-Availability Web Infrastructure using Open-Source Tools

1st Raj Dangi
Dept. of Electrical Engineering and Computer Science
University of Toledo
Toledo, USA
rdangi@rockets.utoledo.edu

2nd Purshottam Singh Thakur
Dept. of Electrical Engineering and Computer Science
University of Toledo
Toledo, USA
pthakur2@rockets.utoledo.edu

***Abstract*—High availability (HA) for web applications is typically achieved through dedicated cloud infrastructure or enterprise hardware, resources often inaccessible to students and small organizations. This paper presents the design and implementation of a fault-tolerant, two-node web system built entirely on commodity hardware using open-source components. The architecture leverages a primary node and a synchronized virtual backup connected via an encrypted overlay network (Tailscale), with a load balancer (HAProxy) managing traffic routing and health monitoring. Experimental results demonstrate a failover response time of approximately 3 seconds and a failback recovery time of 5 seconds, with negligible impact on request latency (0.002s increase). These findings confirm that measurable availability guarantees can be achieved in resource-constrained environments without relying on paid cloud services or complex cluster frameworks.**

***Index Terms*—High availability, web infrastructure, load balancing, overlay networking, HAProxy, Tailscale, LAMP**

I. Introduction

Many entry-level and student-operated web deployments still rely on a single server instance, whether on a local machine or a basic hosting provider. In such settings, service availability is tightly coupled to the health of one machine. Total service disruption can occur due to hardware faults, software misconfiguration, human error, or network outage. Student deployments are particularly vulnerable because they often lack redundant hardware, dedicated administration, or paid failover services.

High availability systems mitigate these risks through redundancy and automated routing, but traditional implementations rely on enterprise tooling or cloud platforms. This work explores whether measurable improvements in availability and fault tolerance can be achieved with commodity hardware available to students. The system consists of a primary server running Arch Linux and a synchronized Ubuntu-based backup running in a virtual machine on the same physical host. This heterogeneity is intentional: it reflects realistic mixed-environment deployments and demonstrates that the design does not depend on a single distribution. Both nodes run the Linux, Apache, MariaDB, and PHP (LAMP) stack. A lightweight encrypted overlay network (Tailscale) enables peer-to-peer communication between nodes without exposing services publicly or requiring conventional VPN configuration.

The research question addressed in this work is: *Can student-scale infrastructure—defined here as low-cost, single-host hardware with one or more local virtual machines and no paid cloud services—achieve measurable availability improvements using open-source tools without enterprise clustering or cloud-native services?*

II. System Architecture

A. Component Overview

The completed system comprises:

- A primary web server running on Arch Linux
- A virtualized backup server running Ubuntu Server
- HAProxy load balancer monitoring both nodes and routing all client traffic
- Secure communication via Tailscale private networking
- Web service stack using Apache, PHP, and MariaDB on both nodes
- Automatic file and database synchronization via cron automation

Tailscale was chosen over traditional VPN or manual SSH tunneling because it provides authenticated, end-to-end encrypted connectivity with minimal configuration and automatically maintains peer reachability even as IP addresses change.

B. Communication Model

Client traffic is routed through HAProxy configured on the primary machine. HAProxy binds to TCP port 80 and forwards HTTP requests to Apache instances running on the primary and backup nodes. It polls both backends using health-check endpoints that return HTTP 200 when the application and database are reachable. If the primary node becomes unavailable or its health check fails repeatedly, HAProxy transparently forwards requests to the backup. When the primary recovers

979-8-3315-7310-2/26/$31.00 © 2026 IEEE

and passes health checks again, traffic is restored without manual intervention.

C. Data Consistency Model

The system implements eventual consistency between primary and backup. File changes propagate using Remote Sync (rsync) every five minutes, and database changes replicate using periodic `mysqldump` export and import every ten minutes. The five-minute file interval was chosen because static assets and templates typically change less frequently than interactive traffic, and rsync is lightweight when few files differ. Database synchronization runs every ten minutes to balance freshness against the higher cost of repeated dumps and restores. Although this approach is not real-time replication, it supports applications tolerant of short propagation delays while keeping implementation complexity low.

D. Node Identity Verification

Each node exposes a dedicated identity endpoint. The primary node returns the string "PRIMARY NODE—ARCH LINUX — adonis," while the backup node returns "BACKUP NODE—UBUNTU VM." These distinct responses make it easy to verify which backend is serving requests during failover tests and to confirm that HAProxy is routing traffic as intended. Figures 1 and 2 illustrate these identity views.

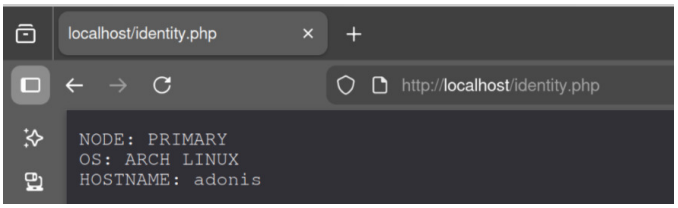

Fig. 1. Identity response from the primary node (Arch Linux) when accessed via `localhost`. The displayed output confirms that HAProxy is routing traffic to the Arch-based backend when it is healthy and reachable.

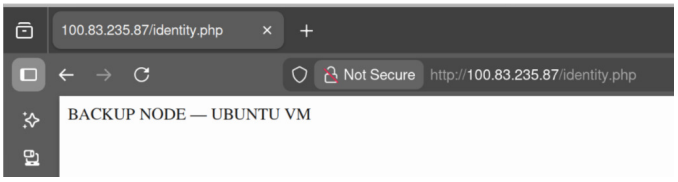

Fig. 2. Identity response from the backup node (Ubuntu VM) when accessed directly through its Tailscale-assigned IP address. The distinct message confirms that the VM is configured as a secondary target and can serve client traffic during failover scenarios.

III. Implementation

A. Primary Node

The primary node runs Apache on `127.0.0.1:8080` while HAProxy binds externally to port 80. The web application is deployed from a Git-backed directory into the Apache document root using `rsync`. Two HTTP endpoints are provided: an identity endpoint that returns a static string and a health endpoint that attempts a MariaDB connection using the application credentials. If the database connection fails, the health endpoint responds with HTTP 500; otherwise it returns HTTP 200 along with a short status message. HAProxy consumes this health endpoint to determine whether the primary node should remain in rotation.

B. Backup Node

The backup node replicates the full LAMP stack with differences only in node identity and in how it is addressed by HAProxy (via its Tailscale IP and port 80). Deployment occurs through automated file transfer over encrypted transport. The same application code and schema are maintained, but the identity endpoint returns a different string to distinguish which node is currently serving traffic. The health endpoint mirrors the primary node's implementation to verify both HTTP reachability and database connectivity on the backup.

C. Overlay Network

Tailscale was chosen over conventional VPNs for its ease of deployment, automatic key management, single sign-on integration, and ability to form a mesh network without manual routing configuration. Each node joins the same Tailscale tailnet and is assigned a stable virtual IP address in the 100.x.x.x range. HAProxy targets these Tailscale addresses when forwarding requests to the backup node, and administrative access to the servers is also restricted to Tailscale, avoiding exposure of SSH or HTTP ports on public interfaces.

D. Load Balancer Configuration

HAProxy performs health polling at three-second intervals with two-failure demotion and two-success promotion. In other words, a backend is marked down after two consecutive failed health checks and marked up again after two consecutive successes. This configuration produced a measured failover interval of approximately three seconds and failback of about five seconds during experiments. Figure 3 displays the configuration used and provides the concrete parameters that govern routing behavior.

E. Synchronization Automation

Two automation scripts execute under cron scheduling:

- File synchronization every five minutes using `rsync` with deletion enabled to mirror the primary web root onto the backup.
- Database synchronization every ten minutes using `mysqldump` on the primary and a corresponding restore into the backup database.

Figures 4 and 5 show log excerpts from the synchronization processes. In the file logs, the timestamps align with the five-minute interval, and the small numbers of bytes transferred confirm that rsync is sending only modified files. In the database logs, recurring entries at ten-minute intervals confirm that periodic dumps and restores are being executed as scheduled.

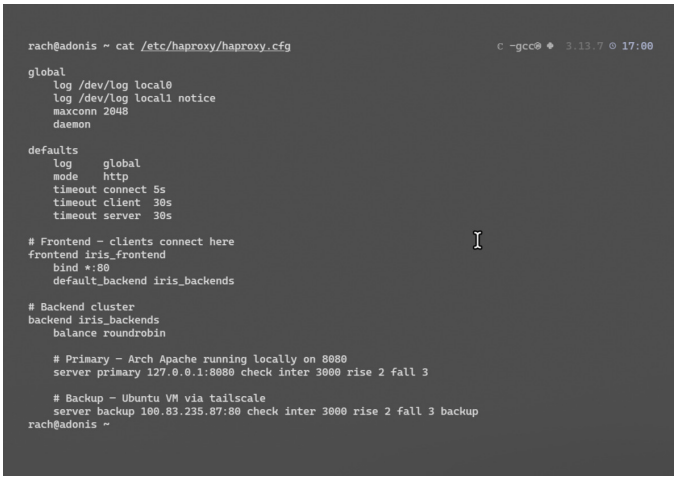

Fig. 3. HAProxy configuration implementing backend health checks and failover policy. The `check` keyword with a three-second interval and `rise`/`fall` thresholds directly determine the observed failover and failback times reported in the evaluation.

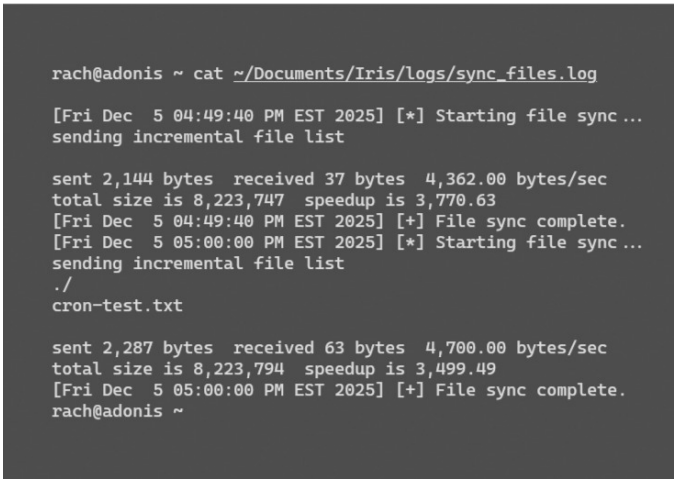

Fig. 4. File synchronization logs on the primary node. The timestamps show executions at five-minute intervals, and the low byte counts indicate that rsync is performing incremental updates, transferring only changed files to keep the backup in sync.

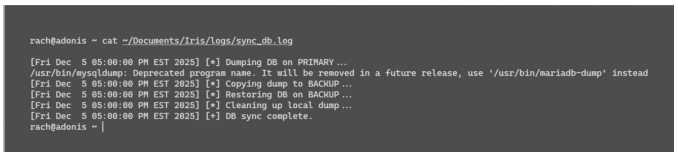

Fig. 5. Database synchronization logs demonstrating recurring `mysqldump` exports and restores at ten-minute intervals. The repeated successful completions indicate that the backup database is being refreshed regularly with the current state of the primary.

IV. Evaluation

Fault injection was performed by shutting down Apache on the primary node while observing service continuity through web requests routed via HAProxy. Figure 6 illustrates that HAProxy reroutes traffic to the backup node immediately after the primary's health checks begin failing. The identity endpoint switches from reporting the primary node string to the backup node string, and the application remains reachable throughout the experiment.

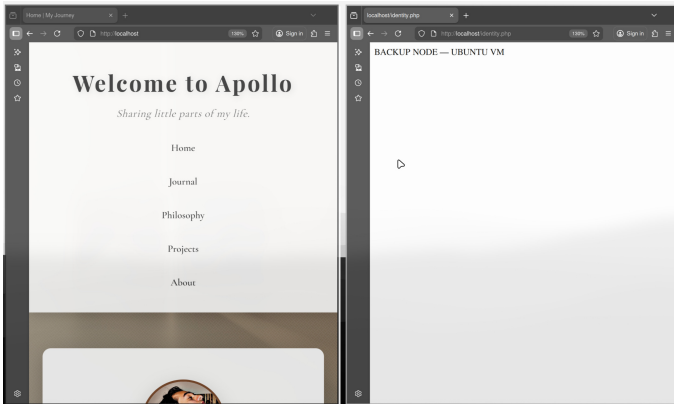

Fig. 6. Failover demonstration. After Apache is stopped on the primary node, the identity view and main website both begin serving from the backup node without manual intervention, showing that HAProxy successfully removed the failed backend from rotation.

Upon restoration of the primary node, routing returns to it, as shown in Figure 7. Once the health endpoint on the primary begins responding with HTTP 200 again, HAProxy reintroduces it into the backend pool and resumes sending new client requests there.

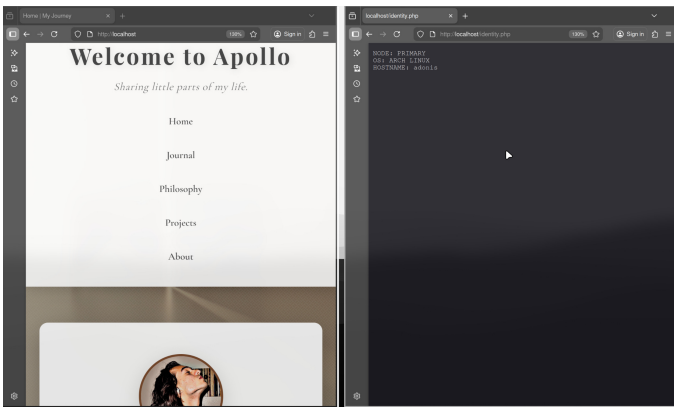

Fig. 7. Failback demonstration. After the primary node is restarted and passes health checks, HAProxy restores routing to the primary backend. The identity endpoint again reports the primary node string, confirming that regular service has resumed.

Response latency was measured using repeated HTTP requests through HAProxy. The average end-to-end latency when serving from the primary node was approximately 0.010 seconds, while the average when serving from the backup node was approximately 0.012 seconds. These values represent the

time to complete a full HTTP request and response, including HAProxy processing and backend handling. The additional 0.002 seconds incurred when routing to the backup reflects the overhead of the extra network hop over Tailscale and virtualized execution, but the increase is small enough to be negligible for typical interactive web browsing.

File modifications became visible on the backup within the five-minute synchronization window, and database values propagated within ten minutes. For applications that can tolerate eventual consistency on the order of several minutes, the system therefore provides both continued service during node failure and reasonably fresh state on the backup. While the experiments focused on short-term failover events, the system was observed to maintain stability over a 24-hour period with periodic synchronizations occurring as scheduled, suggesting suitability for longer-term deployments.

V. Discussion

The presented system achieves student-scale fault tolerance with low cost and moderate implementation effort. By relying only on one physical machine and a virtualized backup, the architecture demonstrates that meaningful availability improvements do not require dedicated clusters or managed cloud platforms. Limitations include eventual consistency rather than synchronous replication, no explicit split-brain prevention in the event of misconfiguration, and the absence of continuous change streaming. The periodic dump-and-rsync approach is simple but may not be suitable for very high write volumes or applications requiring strict transactional guarantees.

As noted, the HAProxy instance itself remains a single point of failure in the current design. Additionally, the ten-minute database synchronization interval may lead to significant data loss in high-write scenarios. Future enhancements may include distributed consensus protocols for leader election, binlog replication for lower-latency database updates and reduced potential data loss in write-intensive applications, dual-primary replication with conflict resolution, floating IP failover to remove dependency on a single HAProxy instance and further strengthen the architecture, or continuous synchronization triggered by filesystem events rather than static intervals.

VI. Conclusion

This study demonstrates that measurable fault tolerance, automated routing, and resilience behavior can be achieved on low-resource infrastructure consisting of a single student-owned machine and a modest virtual machine configuration. The two-node architecture successfully rerouted traffic during failure events with minimal observable latency change and recovered operation autonomously after the primary returned. In direct answer to the research question, the results show that student-scale infrastructure can indeed achieve measurable availability improvements using only open-source tools, without resorting to cloud-native or enterprise clustering services. For students and educators, this experiment offers a practical template for learning about high availability, load balancing, and backup design in environments where dedicated servers and commercial platforms are not available.

Acknowledgment

The authors would like to express their sincere gratitude to the University of Toledo College of Engineering for providing the environment and resources necessary for this research. We specifically thank the Department Chair, Dr. Liang Cheng, and Dr. Ashish Kharel for their guidance, opportunity, and continued support of undergraduate research initiatives. We also thank Anushka Sharad Kumavat (askumavat@owu.edu) for her valuable feedback on the structure, clarity, and presentation of this research, which significantly improved the academic rigor of this work.

References

[1] W. Tarreau, "HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer," [Online]. Available: https://www.haproxy.org/. [Accessed: Feb. 11, 2026].

[2] A. Tridgell and P. Mackerras, "The rsync algorithm," Australian National University, Dept. Computer Science, Tech. Rep. TR-CS-96-05, 1996.

[3] A. Pennarun, "How Tailscale works," Tailscale Blog, Mar. 2020. [Online]. Available: https://tailscale.com/blog/how-tailscale-works/. [Accessed: Feb. 11, 2026].

[4] MariaDB Foundation, "MariaDB Server Documentation," 2024. [Online]. Available: https://mariadb.com/kb/en/. [Accessed: Feb. 11, 2026].

[5] S. S. Gill, A. K. Luhach, and N. Chilamkurti, "Design and Implementation of High-Availability Architecture for IoT-Cloud Services," in *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 10868–10879, Dec. 2019.

[6] A. A. Saif and M. Shafi'i M. A., "A Survey of Dynamic Replication Strategies in Distributed Systems," in *International Journal of Computer Applications*, vol. 110, no. 11, pp. 1–4, 2015.

[7] P. Vixie, "Cron," in *ISC Cron Documentation*, 1993. [Online]. Available: https://ftp.isc.org/isc/cron/. [Accessed: Feb. 11, 2026].

[8] K. Sibbald, "Bacula Main Reference Manual," Bacula.org, Mar. 2017. [Online]. Available: https://www.bacula.org/7.4.x-manuals/en/main/main.pdf. [Accessed: Feb. 11, 2026].

[9] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable internet services," in *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 230–243, 2001.